Boosting In-Memory Computing with Advanced Cache Techniques

Prerana Puttaswamy, Mayur Jain, Shiven Paudyal, Ronak Shukla
Department of Computer Engineering and Computer Science, California State University, Long Beach
Long Beach, CA, USA

Abstract—This survey paper investigates recent advancements in cache memory optimization aimed at propelling in-memory processing capabilities to new heights. Cache memory, a small high-speed volatile memory, stores frequently used data and programs, enabling processors to access data rapidly. Our review examines approaches, designs, and methods to improve cache hierarchies—covering associativity, size, and replacement policies—and their impact on system performance. By surveying state-of-the-art methodologies, this paper lays the groundwork for future advancements in cache memory optimization for in-memory processing.

Index Terms—cache memory, in-memory processing, cache replacement policies, hybrid memory systems, LLC optimization, FPGA, near-memory processing, energy efficiency.

I. INTRODUCTION

Cache memory is a pivotal component in the memory hierarchy of computer systems. In today's rapidly evolving computing landscape, enhancing cache memory has become increasingly critical due to the escalating demand for superior data processing capabilities. As the need for faster and more efficient data handling intensifies, optimizing cache memory emerges as a crucial strategy to support in-memory processing.

This survey conducts an in-depth investigation into recent progress in cache memory optimization, underscoring its critical importance in modern computing. We highlight cache memory's essential role within the memory subsystem hierarchy and examine the necessity of efficient cache optimization techniques to achieve a balanced trade-off between latency, memory interference, and overall system performance.

Our study is framed within the context of contemporary computing paradigms, where in-memory processing is pivotal for deriving insights across big data analytics, machine learning, and real-time processing applications. Through analysis of key research articles, we cover fundamental concepts, challenges, and potential solutions in cache memory optimization.

II. BACKGROUND

The continuous evolution of computer systems has driven the pursuit of maximum efficiency and performance. Early computers faced a significant bottleneck as primary memory could not keep pace with processor speeds, hindering overall performance. Cache memory emerged as a strategic solution to bridge this technological gap, strategically positioned between the CPU and main memory to mitigate the speed disparity [1].

Cache memory acts as a smaller, faster memory bank that stores frequently accessed data and instructions, reducing the need to repeatedly fetch them from the slower main memory. By lowering data access latency, it significantly enhances system performance. Over time, hierarchical cache structures have evolved, comprising L1, L2, and L3 caches, each serving a specific role in optimizing data storage and retrieval processes [4].

Despite its advantages, cache memory deployment has introduced several challenges. Issues such as cache conflicts, misses, evictions, and data consistency across multi-core systems have become critical. Extensive research has addressed these through techniques such as Least Recently Used (LRU) and Least Frequently Used (LFU) replacement policies.

With the advent of in-memory processing paradigms, the focus has shifted to revisiting cache memory configurations to ensure optimization for large-dataset, high-speed workloads. Such optimization is crucial for addressing current high-performance computing challenges and laying the foundation for future advancements [4].

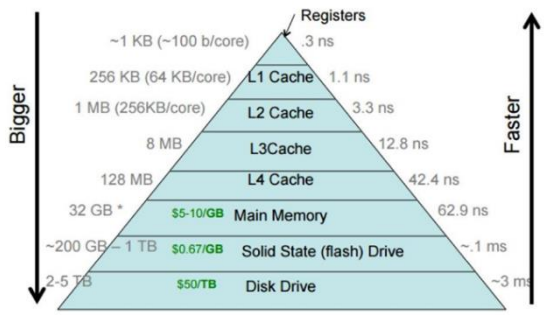

Fig. 1. Cache memory hierarchy illustrating L1, L2, L3 caches and their latency–size trade-offs.

III. MOTIVATION

Enhancing computer system performance is the primary goal of optimizing cache memory for in-memory processing. A major factor propelling advancements is the proliferation of data-intensive applications in fields such as big data analytics, artificial intelligence, and real-time processing. Efficient cache management minimizes latency in retrieving data from slower main memory, enabling faster decision-making—essential in financial trading, healthcare, and autonomous vehicles [1].

Beyond performance, energy efficiency is a compelling motivator. Effective cache management contributes to more sustainable and cost-effective computer systems by reducing reliance on power-hungry main memory. Additionally, industries such as online advertising and e-commerce benefit significantly from improved cache memory via quicker response times and better user experiences.

Optimizing cache memory extends the lifespan of existing hardware. By minimizing the memory access speed gap, organizations can prolong the usability of their current infrastructure. The evolving landscape of memory technologies—including high-bandwidth memory, persistent memory, and non-volatile memory—opens new avenues for cache design and optimization [4].

IV. METHODS

A. Fair Scheduling for Hybrid Memory Systems

This research explores fair memory scheduling and Last Level Cache (LLC) filtering in hybrid memory systems. Memory access behaviors are characterized using five key performance metrics: Misses Per Kilo Cycles (MPKC), Row Buffer Locality (RBL), Bank Level Parallelism (BLP), Read/Write Ratio, and LLC Hit Rate [2].

A Fair Memory Scheduling Model is proposed that prioritizes memory-intensive applications using Average Service Time (AST) as a metric. Applications with higher AST are prioritized to mitigate adverse performance effects. A coordinated cache filtering approach (CMC) considers different memory access patterns—recency-friendly, streaming, thrashing, and random—to identify and filter dead blocks, bypassing the LLC when appropriate [2].

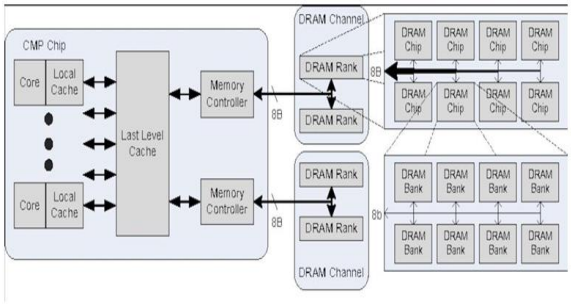

Fig. 2. Hybrid memory architecture with near-memory processing and cache integration.

B. Optimal Cache Utilization via OptiCache

This study introduces OptiCache, a novel offline algorithm that models optimal data placement as a Maximum Cost Minimum Flow (MCMF) network problem. OptiCache optimizes data movement across memory levels to maximize cache utilization in heterogeneous systems comprising DRAM and NVM [3].

The algorithm employs spatial sampling to enhance practicality for large-scale operations. Tested on virtual machines and content distribution networks, OptiCache demonstrated access latency improvements ranging from 8.2% to 44.8% over competing techniques [4].

TABLE I
CACHE TECHNIQUES COMPARISON

Method	Focus Area	Key Metric	Result	Ref.
OptiCache	Data Placement	Access Latency	8.2%–44.8% improvement	[3]
CMC Scheduling	Hybrid Memory	AST / LLC Hit Rate	Balanced fairness	[2]
Write-Buffer	Reliability	AWR-of-Dirty	95.6%–99.8%	[3]
NMP Accelerator	In-Memory Cache	Memory Access Time	61%–83% reduction	[4]
WADD Cache	Energy Efficiency	Energy-Delay Product	13.5% reduction	[7]
MALRU	Hybrid Memory	AMAT / Performance	Up to 22.8% gain	[8]

TABLE I. Summary of Cache Optimization Techniques

C. Write-Buffer Scheme for Multi-Bit Error Protection

Using the gem5 simulator with SPEC CPU2006 benchmarks on an ARM-based processor, this study evaluates a write-buffer scheme for protecting cache memories against multiple-bit errors [3]. The evaluation uses the Access-with-Replica (AWR) metric to assess partial replication reliability, focusing on dirty lines.

Results show substantial improvement over the S-TRB method. With an 8-entry write buffer, the average AWR-of-Dirty is 95.6%, 99.4%, and 99.8% for 1-, 2-, and 4-entry limits respectively. The proposed method also reduces CPI compared to the fully-protected S-TRB-EWB scheme, with optimal design at 16 entries balancing energy overhead and replication capability [3].

D. Near-Memory Accelerator for In-Memory Caching

This approach improves in-memory data caching by integrating an efficient item retrieval mechanism with a near-memory processing (NMP) accelerator [4]. On the software side, ITEM JUMP reduces linked-list iteration, while on the hardware side an NMP-based LSB-first parallel traversal unit accelerates item comparison.

Evaluation results show the hardware accelerator achieving approximately 34× faster item comparison. ITEM JUMP reduces retrieved items by up to 42%, and the NMP-based accelerator reduces memory access time by 61%–83% versus basic traversal units, while reducing overhead by 50%–80% compared to existing methods [4].

E. Array-Specific Dataflow Caches for FPGAs

This study introduces a novel cache architecture for FPGA memory management using the load, compute, store (LCS) paradigm [5]. The system separates off-chip memory accesses into a dedicated dataflow task, with low-level optimizations that minimize DRAM accesses and organize them to align with FPGA memory management challenges.

A throughput-oriented cyclic dataflow protocol enhances HLS tools by ensuring correct hardware functioning. The cache is fully customizable at the C++ source level, offering gains in power, performance, and area, while improving accessibility of FPGA-accelerated applications [5].

F. Cache Assignment on Degraded Broadcast Channels

This research explores strategic cache utilization in K-user broadcast channels to maximize global caching gains [6]. The authors introduce a targeted cache placement strategy that systematically directs content allocation among multiple users.

The study establishes stringent boundaries on the trade-off between capacity and memory, and provides insights into optimal cache allocations for different environmental scenarios. These findings guide strategic cache resource assignment to capitalize on global caching improvements, refining system performance and aiding network optimization [6].

G. Energy-Efficient Cache via Way Access Dominance Detection

This research proposes an energy-efficient shared cache design for chip multiprocessors (CMPs) using Way Access Dominance Detection (WADD) [7]. WADD identifies which cache ways dominate in terms of access, minimizing superfluous lookups while improving hit rates and reducing power usage.

The approach employs way prediction with partial comparisons, delivering a 13.5% decrease in the energy-delay product versus conventional cache architectures. The WADD algorithm dynamically adapts to evolving access patterns and separately pinpoints dominant ways for each core, ensuring correct prediction even under fragmented access patterns [7].

H. Miss Penalty Aware Cache Replacement (MALRU)

Existing cache replacement policies, tailored for DRAM-centric setups, do not perform optimally in hybrid memory environments because miss penalties vary considerably between DRAM and NVM [8]. In response, this paper introduces Average Memory Access Time (AMAT) as a novel metric and proposes a miss penalty-aware LRU-based policy (MALRU).

MALRU gives priority to evicting cache lines with higher miss penalties—notably those linked to NVM—while modifying the LRU queue to maintain compatibility with existing hardware. Empirical evaluations using SPEC CPU2006 benchmarks show MALRU surpasses conventional LRU and hybrid memory-aware partitioning strategies, yielding up to a 22.8% performance improvement [8].

V. CONCLUSION

The rise of data-intensive computing has amplified the need for in-memory processing and enhanced cache memory efficiency. Cache memory serves as a critical mediator between high-speed CPUs and slower main memory.

The methods surveyed demonstrate that advances in fair scheduling, intelligent data placement, error protection, near-memory processing, FPGA-specific caches, energy-aware designs, and penalty-aware replacement policies each contribute meaningfully to system performance. Addressing these challenges

unveils a compelling future where in-memory processing achieves unprecedented speeds while integrating security, scalability, and energy efficiency.

Anticipating next-generation computer systems equipped for escalating data-intensive demands, these advancements have the potential to transform industries and shape the future of computing.

VI. FUTURE CONSIDERATIONS

Looking ahead, transformative breakthroughs in cache memory enhancement are on the horizon. Machine learning algorithms applied to predict data access patterns and dynamically adjust cache strategies open promising avenues in AI-driven cache management.

Hybrid cache memory systems combining technologies beyond DRAM, SRAM, and NVM offer customized solutions for diverse workloads. Security-aware cache architectures address vulnerabilities in cloud computing and sensitive data environments, ensuring data confidentiality and integrity.

Cache optimization for emerging applications—edge computing, the Internet of Things (IoT), and artificial intelligence—is part of this progression. An adaptive approach allows cache designs and algorithms to evolve with the dynamic computing requirements of these paradigms.

. REFERENCES

[1] M. Kee, C. Han and G.-H. Park, "An Integrated Solution to Improve Performance of In-Memory Data Caching With an Efficient Item Retrieving Mechanism and a Near-Memory Accelerator," IEEE Access, vol. 11, pp. 78726–78736, July 2023.

[2] P. Li, C. Pronovost, W. Wilson, B. Tait, J. Zhou, C. Ding, and J. Criswell, "Beating OPT with Statistical Clairvoyance and Variable Size Caching," in Proc. ASPLOS, 2019, pp. 243–256.

[3] Y. Chi, L. Guo, Y.-K. Choi, J. Wang, and J. Cong, "Extending High-Level Synthesis for Task-Parallel Programs," in Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays, Feb. 2021, pp. 204–213.

[4] L. Zhang, R. Karimi, I. Ahmad, and Y. Vigfusson, "Optimal Data Placement for Heterogeneous Cache, Memory, and Storage Systems," June 2020.

[5] X. Xie, L. Xiao, D. Wei, Q. Li, Z. Song, and X. Ge, "Pinpointing and Scheduling Access Conflicts to Improve Internal Resource Utilization in Solid-State Drives," Front. Comput. Sci., vol. 13, no. 1, pp. 35–50, 2019.

[6] G. Brignone, M. U. Jamal, M. T. Lazarescu, and L. Lavagno, "Array-Specific Dataflow Caches for High-Level Synthesis of Memory-Intensive Algorithms on FPGAs," IEEE Access, vol. 10, pp. 118858–118877, Nov. 2022.

[7] J. Chen et al., "HotRing: A Hotspot-Aware In-Memory Key-Value Store," in Proc. 18th USENIX Conf. File Storage Technol. (FAST), Feb. 2020, pp. 239–252.

[8] S. Saeedi Bidokhti, M. Wigger, and A. Yener, "Benefits of Cache Assignment on Degraded Broadcast Channels," IEEE Trans. Inf. Theory, vol. 65, no. 11, pp. 6999–7019, Nov. 2019.

[9] H. Jin, D. Chen, H. Liu, X. Liao, R. Guo, and Y. Zhang, "Miss Penalty Aware Cache Replacement for Hybrid Memory Systems," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 39, no. 12, pp. 4669–4682, Dec. 2020.